Proiect MG3D

Roșu Mihai Cosmin

a. API 3D

Am ales să folosesc OpenGL pentru implementarea proiectului.

Pentru gestiunea ferestrei am folosit biblioteca GLFW (versiunea 3.4). Cu ajutorul acesteia am creat fereastra principală cu care se poate interacționa și vizualiza scena.

De asemenea, am folosit și bilbioteca GLAD care oferă o multitudine de macro-uri pentru interacțiunea cu OpenGL.

Structura programului este următoarea:

- La început are loc faza de inițializare în care este creată fereastra principală (GLFWwindow) cu rezoluția default (în momentul de față este 1200x900) și este setat viewport-ul.
- Apoi, sunt setate callback-urile dintre care, pentru acest punct este relevant FramebufferSizeCallback, care permite redimensionarea ferestrei în timpul rulării programului.
- Mai departe sunt inițializate și importate clasele și asset-urile necesare.
- După aceea, urmează bucla principală care rulează până la închiderea programului.
    - În cadrul buclei, se colectează input-ul care permite apăsarea tastei Escape pentru închiderea programului.
    - Întrucât fiecare iterație a buclei principale reprezintă un cadru, trebuie curățate bufferele ce sunt folosite pentru randare. Acest lucru se face prin glClearColor (setează culoarea default) și glClear (curăță bufferele specificate).
    - Urmează partea în care sunt configurate matricele de transformare pentru obiecte, iar apoi are loc randarea acestora folosind shaderele specificate.

    - La finalul iterației, sunt trimise bufferele folosite pentru a fi desenate pe ecran folosind glfwSwapBuffers, și este apelată și funcția glfwPollEvents pentru a colecta evenimentele folosite pentru input.
- Dacă bucla principală este oprită, programul este închis cu glfwTerminate.

b. Format intern propriu de Mesh-uri pentru obiecte

Pentru a îmi reprezenta propriul format de mesh-uri, am creat clasa Mesh. De asemenea am creat o structură VertexData care va fi folosita pentru a stoca mai ușor datele vârfurilor unui obiect (poziția, normala, coordonate de textură și culoare).

În cadrul clasei Mesh sunt stocate diferite informații despre mesh, precum: un id (string), un vector de VertexData pentru vârfuri, un vector de întregi pentru indici, un GLuint pentru VAO, un GLuint pentru VBO, un GLuint pentru EBO și un GLenum pentru modul de desenare (în momentul de față este folosit doar GL_TRIANGLES).

Funcțiile principale oferite de această clasă sunt:

- Init – funcție care inițializează informațiile mesh-ului, primind ca argumente vectorii de vârfuri și de indici. Aceasta creează VAO-ul și VBO-ul, în care stochează vârfurile primite în ordinea: poziție, normală, coordonate textură, culoare. Apoi, creează EBO-ul în care stochează indicii.
- Render – funcție care desenează mesh-ul folosind macro-ul glDrawElements.

Pentru a testa funcționalitatea clasei Mesh, am creat manual un cub cu fețe de culori diferite.

Pentru desenarea cubului este necesar un shader, astfel, pentru convenienței, am creat o clasă Shader.

Pentru simplitate, pe parcursul proiectului shaderele vor avea urmatoarea structură pentru nume: <nume>.VS.glsl pentru vertex shader și <nume>.FS.glsl pentru fragment shader. Pentru ambele, <nume> trebuie să fie același string.

În cadrul clasei sunt stocate informații precum un string identificator și un GLuint pentru id-ul programului.

Funcțiile puse la dispoziție de această clasă sunt:

- CreateProgram – funcție care primește ca argument un string pentru <nume>. Această funcție preia codul din vertex shader și fragment shader și creează câte un shader pentru fiecare folosind macro-urile glShaderSource și glCompileShader. În final este creat un program ce are cele 2 shadere atașate.
- Use – funcție care apelează macro-ul glUseProgram dacă id-ul programului din clasă este diferit de 0.
- SendUniform<tip> - funcții care trimit date de tipul <tip> ca uniform în shader.

Astfel, am folosit un shader simplu pentru a desena cubul pe ecran.

c. Importer pentru un format cunoscut de obiecte 3D

Pentru a importa obiecte 3D am folosit Assimp (versiunea 6.0.2).

Înainte de a implementa logica pentru importare, este necesară crearea unei clase pentru texturi, Texture. Pentru încărcarea imaginilor a fost folosit stb_image.

Această clasă stochează informații despre textură precum: un string pentru nume, întregi pentru lățimea imaginii, înălțimea imaginii și numărul de canale, un GLuint pentru id-ul texturii, un GLuint pentru tipul texturii (default este GL_TEXTURE_2D) și GLenum-uri pentru tipul de wrap, tipul de min filter, tipul de mag filter și modul de culoare.

Principalele funcții ale clasei sunt:

- GenerateTexture – funcție care creează o textură folosind macro-uri și setează filtrele și modul de wrap.
- LoadTexture – funcție care primește ca argumente path-ul către o imagine și un bool care specifică dacă să fie flipped imaginea la încărcare. Această funcție încarcă imaginea, setează modul de culoare și generează un mipmap.
- BindToActiveTexture – funcție care primește ca argument un GLenum pentru a specifica texture unit-ul la care să fie legată textura. Funcția folosește macro-urile glActiveTexture și glBindTexture pentru a îndeplini funcționalitatea.

- Unbind – funcție care face bind la id-ul de textură 0.

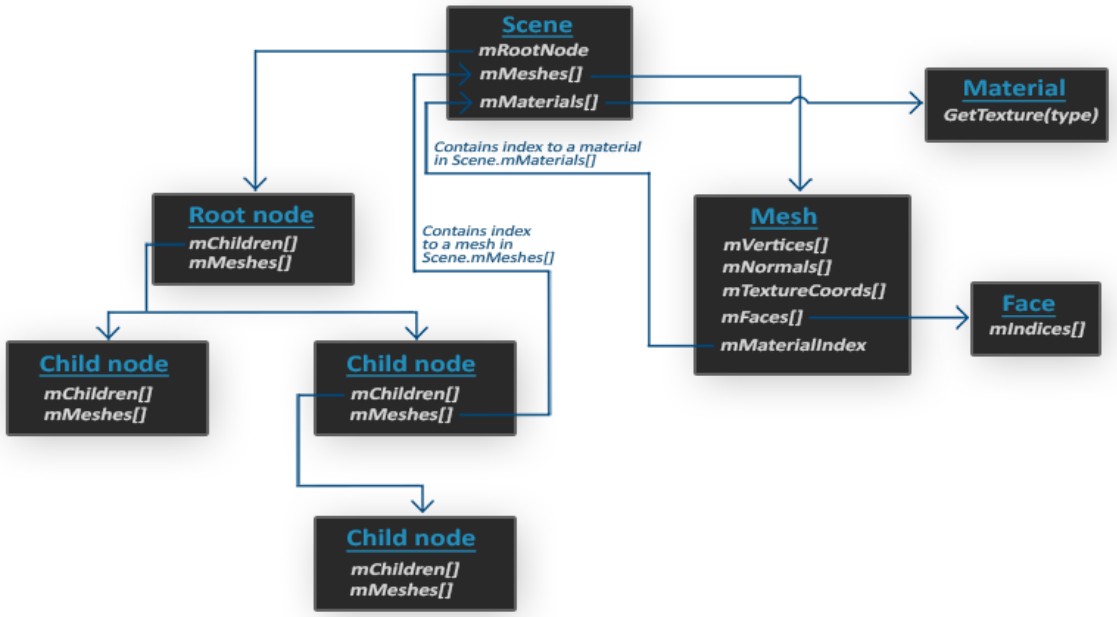
Pentru a folosi Assimp, este necesară modificarea clasei Mesh.

Mai întâi a fost adăugată o clasă Material care să stocheze informații despre materiale, precum: culoare ambientală, culoare difuză, culoare speculară, culoare emisivă, factor de strălucire și pointer către o textură.

Mai departe, pentru a simplifica importarea obiectelor 3D, a fost creată încă o clasă, MeshEntry, care stochează numărul de indici ai mesh-ului, index-ul primului vârf (va fi folosit pentru a căuta în lista completă de vârfuri), index-ul primului indice, index-ul materialului folosit.

În cadrul clasei Mesh, au fost adăugate: un vector de MeshEntry-uri, un vector de materiale, un bool care specifică dacă mesh-ul a fost importat, câte un vector de vec3-uri pentru pozițiile, normalele și coordonatele de textură ale vârfurilor, încă 2 VBO-uri și un string ce stochează directorul în care se află obiectul 3D importat.

Assimp importează obiectele sub forma unei scene formate din mai multe mesh-uri. Fiecare MeshEntry reprezintă un mesh din cadrul scenei importate.



Structura claselor folosite de Assimp

Ca funcții au fost adăugate/modificate următoarele:

- Load – funcție care primește 2 argumente de tip string, unul pentru path-ul către directorul ce conține obiectul, iar unul pentru numele fișierului cu obiectul. Funcția folosește importer-ul de la Assimp pentru a citii fișierul și a obține datele sub forma unei clase de tip aiScene. De asemenea, la importare sunt folosite următoarele flag-uri: aiProcess_GenSmoothNormals, aiProcess_FlipUVs, aiProcess_Triangulate. După importare, este apelată funcția InitScene.
- InitScene – funcție care inițializează mesh-ul primind ca argument un aiScene. Pentru fiecare mesh din cadrul scenei este creat un MeshEntry și populat cu informațiile necesare. Apoi pentru fiecare mesh este apelată funcția InitMesh. În final sunt apelate funcțiile InitMaterials și InitImported.
- InitMesh – funcție care inițializează mesh-ul parțial pornind de la un aiMesh primit ca argument. Pentru fiecare vârf al aiMesh-ului, sunt adăugate poziția, normala și coordonatele de textură la vectorul respectiv. Apoi sunt adăugați indicii folosind fețele (aiFace) din cadrul aiMesh-ului.
- InitMaterials – funcție care inițializează materialele din cadrul unei scene (aiScene) primite ca argument. Pentru fiecare aiMaterial din scenă, este creată o instanță de Material care este populată cu culorile din aiMaterial. Dacă există și o textură pentru material, este importată prin crearea unei instanțe de Texture din care este apelată funcția Load cu path-ul texturii.
- InitImported – funcție care generează și populează bufferele necesare mesh-ului. Mai întâi creează VAO-ul, apoi 3 VBO-uri, fiecare primind câte o informație despre vârfuri (poziție, normală, respectiv coordonate de textură), și apoi EBO-ul, care primește indicii mesh-ului.
- Render – funcția acum apelează RenderImported dacă mesh-ul a fost importat, altfel apelează RenderSimple care reprezintă vechea funcție de render.
- RenderImported – funcția randează mesh-ul folosind vectorul de MeshEntry-uri. Pentru fiecare, dacă materialul corespunzător are textură, îi face bind la texture unit-ul 0 și folosește macro-ul glDrawElementBaseVertex care primește index-ul primului vârf din lista de vârfuri și index-ul primului indice, pentru a randa doar MeshEntry-ul respectiv.

Pentru testarea texturii am creat un shader care să folosească o textură și am folosit cubul creat anterior căruia i-am atribuit o textură de crate. Pentru testarea importer-ului, am folosit modelul 3D “bamboo” din cadrul framework-ului de la PPBG, acesta având și o textură.

d. Modul de 3D Math

Pentru modulul de 3D math am folosit biblioteca glm, întrucât oferă structuri de date (vec3, mat4, etc.) și funcționalități (translate, rotate, scale) ce se potrivesc ceea ce era necesar pentru OpenGL.

e. Gestiune interacțiune cu utilizatorul

Pentru gestiunea interacțiunii cu utilizatorul am create clasele Keyboard și Mouse.

Clasa Keyboard stochează informații despre tastele apăsate (un vector de bool-uri unde indicii reprezintă tastele - keys) și tastele care tocmai si-au modificat starea (același format - keysChanged).

De asemenea, oferă un callback ce primește o tastă și o acțiune atribuită ei. Folosind această acțiune, o tastă poate deveni apăsată sau, dacă si-a schimbat starea, este actualizat vectorul de keysChanged.

Clasa oferă următoarele funcții:

- Key – funcție care primește o tastă și întoarce true dacă tasta este apăsată, sau false altfel.
- KeyChanged – funcție care primește o tastă și întoarce true dacă tasta și-a schimbat starea. De asemenea, setează keysChanged pe false.
- KeyWentDown – funcție care verifică dacă o tastă tocmai a fost apăsată prin verificarea vectorului keys și a vectorului keysChanged.
- KeyWentUp – funcție care verifică dacă o tastă tocmai a fost eliberată.

Clasa Mouse este similară cu Keyboard. Aceasta stochează informații despre poziția absolută a cursorului (atât în cadrul curent, cât și în precedentul), diferența de poziție față de cadrul precedent, diferența de scroll față de cadrul precedent, un vector cu starea butoanelor mouse-ului și unul cu schimbarea de stare, în mod similar cu logica din Keyboard.

De asemenea, clasa oferă 3 callback-uri, unul pentru poziția cursorului, unul pentru butoanele mouse-ului și unul pentru scroll.

Funcțiile oferite (Button, ButtonChanged, ButtonWentDown, ButtonWentUp) sunt referitoare la butoanele mouse-ului și funcționează la fel ca în clasa Keyboard.

În programul principal, sunt setate callback-urile folosind funcții din GLFW.

f. Implementare Cameră de tip FPS

Pentru implementarea camerei am folosit o clasă Camera.

Aceasta stochează informații despre cameră precum: un vec3 pentru poziție, câte un vec3 pentru fiecare din cele 3 direcții (up, forward și right), câte un vec3 pentru fiecare din cele 3 direcții globale (world), două float-uri pentru orientare (yaw. pitch), un float pentru field of view, un float pentru zNear și un float pentru zFar.

Când o instanță de Camera este creată, primește ca argument un vec3 pentru poziție.

Clasa oferă funcții precum:

- Rotate – funcție care rotește camera prin modificarea valorilor yaw și pitch cu valorile primite ca argument. De asemenea, se apelează funcția internă UpdateCameraVectors.
- Move – funcție care modifică poziția camerei folosind argumentele primite (direcție și cantitate).
- Zoom – funcție care modifică fov-ul folosind argumentul primit.
- GetViewMatrix – funcție care întoarce matricea view prin apelarea funcției lookAt din glm.
- UpdateCameraVectors – funcție internă care actualizează vectorii de direcție ai camerei folosind valorile de yaw și pitch.

În cadrul programului principal, camera este creată la început. În bucla principală există funcția de ProcessInput, unde folosind tastele WASDQE și clasa Keyboard este modificată poziția camerei, și folosind mouse-ul și clasa Mouse este rotită camera.

De asemenea, pentru mișcarea camerei este folosită o valoare de deltaTime, calculată prin diferența de timp absolut din cadrul curent față de cel precedent.

g. Model iluminare, material și texturi

Am ales să implementez modelul de iluminare Phong. În cadrul acestui model există mai multe componente de iluminare: componenta emisivă, componenta ambientală, componenta difuză și componenta speculară. Pentru proiect am folosit toate componentele cu excepția celei emisive.

Fiecare obiect are un material propriu ce este folosit pentru specificarea culorii în cadrul componentelor de iluminare.

Pentru a putea calcula componentele de iluminare, sunt necesare surse de lumină. În cadrul proiectului există 2 tipuri de surse de lumină, o lumină punctiformă și o lumină de tip spot. De asemenea, există și o lumină globală.

Pentru fiecare obiect, sunt trimise către shader materialul, mai multe liste cu informații despre luminile din scenă, lumina globală și poziția camerei. Datorită implementării shader-ului, există o limită de 10 surse de lumină din fiecare tip.

Pentru calcularea iluminării se calculează iluminarea fiecărei surse de lumină care apoi este atenuată prin înmulțirea cu un factor de atenuare. Acest factor depinde de distanță pentru luminile punctiforme, și de unghi pentru luminile de tip spot. De asemenea, pentru luminile de tip spot, dacă unghiul dintre lumină și punct este prea mare, acel punct nu va primi deloc iluminare. În final toate componentele de iluminare sunt adunate.

Pentru texturare, culorile din materialul obiectului sunt înlocuite de textură prin folosirea funcției texture2D.

h. Scene Management

Pentru management-ul scenei, am restructurat o parte din cod. Pentru început am creat o clasă GameObject ce este folosită pentru fiecare obiect din scenă. În cadrul clasei sunt stocate informații precum: poziția obiectului, rotația obiectului, scala obiectului, matricea model și referințe la mesh, shader, material sau clasa de lumină.

Clasa oferă și funcția CalculateModelMatrix, care folosește poziția, rotația și scala stocate pentru a calcula matricea model în următoarea ordine: scala -> rotația -> poziția. Totuși, matricea model din cadrul clasei este publică astfel încât să poată fi modificată în mod direct dacă se dorește obținerea unui efect mai complex.

Pentru material a fost refolosită clasa Material care fusese creată pentru mesh-urile importate cu Assimp.

Clasa Light este folosită pentru obiectele ce sunt surse de lumină. Aceasta stochează tipul sursei de lumină (Point sau Spot), culoarea luminii și, în cazul luminilor spot, direcția și unghiul luminii.

Mai departe, am creat o clasă Scene care reprezintă clasa de bază pentru scene, stocând informații despre fereastra aplicației, lumina globală, deltaTime și referințe la cameră, mesh-uri, shadere, texturi, elemente UI și obiectele din scenă.

Funcțiile importante oferite de clasă sunt:

- Init – funcție care inițializează contextul pentru GLFW.
- Update – funcție ce conține un loop cu logica pentru fiecare cadru, ce rulează până când funcția glfwWindowShouldClose întoarce 1.
- StartFrame – funcție ce este apelată la începutul fiecărui cadru. Are ca scop curățarea bufferelor de culoare și adâncime.
- UpdateLogic – funcție pur virtuală ce are ca scop conținerea întregii logici pentru fiecare cadru.
- EndFrame – funcție apelată la finalul fiecărui cadru. Folosită pentru trimiterea bufferelor pentru a fi afișate pe ecran.
- Terminate – funcție care închide contextul ImGui și GLFW.
- SaveScene – funcție care salvează toate informațiile din cadrul scenei într-un fișier XML folosind biblioteca tinyxml2.
- LoadScene – funcție care populează scena pe baza informațiilor dintr-un fișier XML.

Pentru a implementa logica scenei, este creată încă o clasă, SimpleScene, care extinde clasa Scene și următoarele funcții:

- Init – funcția extinsă creează fereastra, inițializează bilbioteca GLAD, setează callback-urile, inițializează biblioteca ImGui și creează conținutul scenei sau încarcă acest conținut folosind LoadScene.
- StartFrame – funcția extinsă calculează deltaTime-ul și apelează ProcessInput. De asemenea, începe un cadru nou pentru ImGui.
- UpdateLogic – funcția implementează logica din fiecare cadru, mai exact calcularea matricilor model pentru fiecare obiect, trimiterea informațiilor necesare la shader și randarea obiectelor. De asemenea sunt randate și elementele UI.
- EndFrame – funcția extinsă desenează elementele UI din buffer-ul ImGui.

Pentru salvarea și respectiv încărcarea scenei, a fost folosit un format XML propriu.

Primul element este de tip Scene și are atributele screen_resolution, screen_bg_color și global_light.

În cadrul elementului de scenă se află elementul de tip Camera care are atribute precum position, orientation, fov, zNear și zFar.

În continuare, în elementul scenei se afla un element de tip Meshes. Acest element are un atribut count și conține câte un element de tip Mesh pentru fiecare mesh relevant scenei. Elementele de tip Mesh sunt de 2 tipuri. Fie conțin multiple elemente de tip Vertex, fiecare cu atribute precum position, normal, texCoords și color, și multiple elemente de tip Index cu atributul indices, fie conțin atributele directory și file în cazul în care sunt obiecte încarcate prin Assimp.

În continuare se află elementul de tip Shaders cu atributul count. Acest element conține câte un element de tip Shader pentru fiecare shader relevant scenei, acest element având atributul path cu calea către shadere.

Mai departe, se află elementul Textures cu atributul count, care conține câte un element Texture pentru fiecare textură încărcată separat, relevantă scenei. Acest element conține atributul path cu calea către imagine.

Apoi, urmează elementul UIElements cu atributul count. Acesta conține câte un element de tip UIElement pentru fiecare element UI din scenă. Acest element conține un atribut position și un atribut uiType ce specifică tipul de element UI. În

funcție de tip, diferă restul atributelor, acestea fiind fie text, fie size, fie diferite culori relevante elementului UI. Pentru elementul de tip dropdown, există în interiorul acestuia mai multe elemente de tip DropdownItem pentru fiecare intrare din lista dropdown-ului.

Ultimul element din elementul scenei este cel de tip GameObjects cu atributul count. Acest element conține câte un element de tip GameObject pentru fiecare obiect din cadrul scenei. Acest element conține atribute precum mesh și shader ce specifică mesh-ul și shader-ul folosite. Mai mult, fiecare GameObject conține un element de tip Transform care are atributele position, rotation și scale. În funcție de obiect, GameObject-ul mai poate conține un element de tip Material cu cele 4 culori: ambientală, difuză, speculară și emisivă, iar, dacă este cazul, poate conține și o textură. Dacă obiectul este o sursă de lumină, GameObject-ul va conține un element de tip Light cu atributele lightType și color. Dacă lightType-ul este Spot, va mai conține și atributele direction și angle.

Pentru încărcarea scenei, se parsează documentul XML element cu element și sunt create clasele corespunzătoare pentru fiecare. Procesul este practic inversul celui de salvare a scenei.

i. 2D GUI System: text, meniuri, butoate, liste, edit boxes, etc.

Pentru sistemul de UI am ales să folosesc biblioteca DearImGui fiindcă oferă multe elemente de UI deja implementate și ușor configurabile.

Pentru partea de setup, trebuie ca la începutul programului sa fie creat un context pentru ImGui, iar în fiecare cadru la început să fie creat un cadru nou pentru ImGui, iar la final să fie desenate pe ecran toate elementele din bufferele ImGui.

Pentru elementele de UI, am creat o clasă de bază UIElement care stochează poziția elementului pe ecran și doua variabile de tip bool care specifică dacă elementul este activat sau accesat. Clasa oferă o singură funcție care este pur virtuală și anume funcția de Render.

Am introdus 5 tipuri de elemente UI: text simplu, slider, text de input (editabil din joc), buton și dropdown.

Pentru textul simplu am ales să îl creez desenând direct în buffer-ul ImGui, fără a folosi un widget ImGui deja existent. Motivul a fost faptul că orice configurare i-aș fi făcut, rămânea o mică margine în jurul widget-ului, ceea ce nu îmi doream.

Așadar, am creat clasa Text care extinde clasa UIElement. Aceasta stochează textul din element, culoarea textului și dimensiunea fontului.

Funcțiile implementate sunt:

- UpdateText – actualizează textul din elementul UI.
- SetColor – actualizează culoarea textului.
- Render – adaugă textul cu culoarea specificată și cu dimensiunea fontului aleasă direct in ForegroundDrawList.

Pentru slider, similar cu textul simplu, am ales să îl desenez direct în buffer, din același motiv, plus motivul că în widget-ul de slider din ImGui, nu se putea face ușor un efect de umplere pentru slider. Alegerea asta are ca dezavantaj faptul că nu se poate interacționa direct cu slider-ul.

Am creat clasa Slider ce extinde clasa UIElement și stochează valoarea slider-ului (0-1), dimensiunea slider-ului și 2 culori pentru fundal și umplere.

Funcțiile implementate sunt:

- UpdateValue – actualizează valoarea slider-ului.
- GetValue – întoarce valoarea slider-ului.
- Render – desenează 2 dreptunghiuri direct în ForegroundDrawList folosind cele 2 culori.

Pentru textul de input am ales să folosesc widget-ul InputText din ImGui deoarece avea implementată funcționalitatea de editare.

Am creat clasa InputText ce extinde UIElement și stochează dimensiunea widget-ului, o culoare de fundal și una a textului și un buffer pentru text.

Funcțiile implementate sunt:

- GetText – întoarce textul stocat in input text.
- Render – creează un window ImGui cu widget-ul InputText folosind culorile și dimensiunea precizate.

Pentru buton, similar cu textul de input, am ales să folosesc widget-ul Button din ImGui.

Am creat clasa Button care extinde UIElement și stochează dimensiunea widget-ului, textul de pe buton, un callback ce este apelat la apăsarea butonului și 4 culori relevante butonului.

Funcțiile implementate sunt:

- ChangeColors – actualizează culorile butonului.
- SetCallback – setează callback-ul butonului.
- Render – creează un window ImGui cu widget-ul Button folosind culorile și dimensiunea precizate. Folosing widget-ul Button se apelează callback-ul butonului.

Pentru dropdown, similar cu butonul și textul de input, am folosit widget-ul Selectable din ImGui împreună cu funcționalitatea de Combo din ImGui pentru a afișa elementul ales.

Am creat clasa Dropdown ce extinde UIElement și stochează dimensiunea widget-ului, o lista de elemente, indicele elementului ales și 5 culori relevante dropdown-ului.

Funcțiile implementate sunt:

- ChangeColors – actualizează culorile dropdown-ului.
- ClearItems – eliberează lista de elemente.
- AddItem – adaugă un element în listă.
- AddItems – adaugă mai multe elemente în listă.
- GetSelected – întoarce elementul selectat din listă.
- Render – creează un window ImGui cu funcționalitatea de Combo, iar apoi creează un widget Selectable folosind dimensiunea și culorile precizate.